

# Projektdokumentation mit Handout

## Pygame-Projekt: Wechsel zwischen einzelnen Spielen innerhalb eines Fensters

|                           |                                                                       |
|---------------------------|-----------------------------------------------------------------------|
| <b>Projektart</b>         | Modulares Pygame-Projekt mit mehreren Sub-Spielen                     |
| <b>Kernthema</b>          | Zentrale Spielverwaltung, Wechsel per ENTER, gemeinsame Einstellungen |
| <b>Sprache/Bibliothek</b> | Python mit Pygame                                                     |
| <b>Bestandteile</b>       | main.py, settings.py, base_game.py, games/-Package                    |
| <b>Zielgruppe</b>         | Ausbildung / Unterricht / Projektvorstellung                          |
| <b>Autoren</b>            | Marco Gilg, Maximilian Ziegler (github: max9by)                       |

**Kurzbeschreibung:** Das Projekt zeigt, wie mehrere kleine Spiele nicht in getrennten Fenstern, sondern innerhalb eines einzigen Pygame-Fensters ausgeführt werden können. Ein zentraler Manager entscheidet, welches Spiel gerade aktiv ist, sammelt die Events und ruft für das aktive Spiel jeweils `handle_events()`, `update()` und `draw()` auf.

### Inhaltsübersicht

- 1. Projektidee und Zielsetzung
- 2. Projektstruktur und Paketaufbau
- 3. Zentrale Klassen und Verantwortlichkeiten
- 4. Ablauf des Spielwechsels
- 5. Bewertung der Paketstruktur und `__init__.py`
- 6. Erweiterbarkeit des Projekts
- 7. Mögliche Verbesserungen
- 8. Handout für die Abgabe / Präsentation

## 1. Projektidee und Zielsetzung

Ziel des Projekts ist der Aufbau einer modularen Pygame-Anwendung, in der mehrere einzelne Spiele innerhalb eines gemeinsamen Fensters laufen. Statt jedes Spiel als eigenständiges Programm mit eigener Pygame-Schleife zu starten, übernimmt eine zentrale Klasse die Verwaltung. Dadurch bleibt das Fenster erhalten, und nur die aktive Spielklasse wird gewechselt.

Der wichtigste technische Schwerpunkt liegt in der Trennung der Verantwortlichkeiten: Der Manager steuert Fenster, Hauptschleife und Spielwechsel; die einzelnen Spiele kümmern sich nur um ihre eigene Eingabe, Logik und Darstellung. Gemeinsame Daten wie Bildschirmgröße, FPS, Goldmünzen, Leben oder Level werden in einer Settings-Klasse gebündelt.

**Leitfrage:** Wie kann man mehrere kleine Spiele so strukturieren, dass sie in einem gemeinsamen Pygame-Fenster laufen und über eine Taste gewechselt werden können?

## 2. Projektstruktur und Paketaufbau

Die Projektstruktur ist in Hauptdateien und ein Unterpaket für die Spiele aufgeteilt. Die Hauptdateien liegen direkt im Projektordner. Die konkreten Spielmodule befinden sich im Unterordner games/.

### Projektstruktur

```

projekt_ordner/
├── main.py           # Manager, Hauptschleife und Spielwechsel
├── settings.py       # Globale Einstellungen und gemeinsame Spielwerte
├── base_game.py      # Gemeinsame Basisklasse für alle Spiele
└── games/            # Package für einzelne Spiele
    ├── __init__.py   # Kennzeichnet den Ordner als Package / optionale Exporte
    ├── game_one.py   # Erstes Spiel
    ├── game_two.py   # Zweites Spiel
    └── game_three.py # Optionales weiteres Spiel
  
```

| Datei/Ordner      | Aufgabe                           | Wichtige Inhalte                                           |
|-------------------|-----------------------------------|------------------------------------------------------------|
| main.py           | Startpunkt und zentrale Steuerung | SuperManager, Hauptschleife, Event-Sammlung, Spielwechsel  |
| settings.py       | Zentrale Projektwerte             | Fenstergröße, FPS, gemeinsame Werte wie gold_coins         |
| base_game.py      | Gemeinsame Oberklasse             | Einheitliche Methoden: handle_events(), update(), draw()   |
| games/            | Sammlung der Sub-Spiele           | Jedes Spiel liegt in einer eigenen Datei                   |
| games/__init__.py | Package-Datei                     | Macht den Ordner importierbar und kann Klassen exportieren |

## 3. Zentrale Klassen und Verantwortlichkeiten

### 3.1 SuperManager in main.py

Die Klasse SuperManager ist der zentrale Manager des Projekts. Sie initialisiert Pygame, erzeugt das Fenster, lädt die gemeinsamen Einstellungen, erstellt die Spielobjekte und führt die Hauptschleife aus. In game\_list werden die einzelnen Spiele als Objekte gespeichert. current\_game\_index bestimmt, welches Spiel gerade aktiv ist.

#### Auszug: Spielobjekte als Liste

```
self.game_list = [  
    GameOne(self.screen, self.settings),  
    GameTwo(self.screen, self.settings),  
]  
self.current_game_index = 0
```

Der Vorteil dieser Lösung ist, dass der Manager nicht ständig neue Fenster öffnet. Das aktive Spiel wird nur durch den Index in der Liste bestimmt. Dadurch kann die Anwendung später einfach um weitere Spiele erweitert werden.

### 3.2 BaseGame als gemeinsame Schnittstelle

Die Klasse BaseGame ist die Oberklasse für alle Sub-Spiele. Sie speichert screen und settings und definiert die Methoden, die jedes Spiel besitzen soll. Die Methoden sind zunächst leer und werden in den Unterklassen überschrieben.

#### Vereinfachter Aufbau der Basisklasse

```
class BaseGame:  
    def __init__(self, screen, settings):  
        self.screen = screen  
        self.settings = settings  
        self.active = True  
        self.next_game = None  
  
    def handle_events(self, events):  
        pass  
  
    def update(self):  
        pass  
  
    def draw(self):  
        pass
```

**Prinzip:** Alle Spiele besitzen die gleiche Schnittstelle. Der Manager muss deshalb nicht wissen, welches konkrete Spiel aktiv ist. Er ruft immer dieselben Methoden auf.

### 3.3 SuperSettings als gemeinsamer Zustand

SuperSettings enthält zentrale Einstellungen und spielübergreifende Werte. Dazu gehören die Fenstergröße, die Bildrate und gemeinsame Spieldaten. Weil alle Spiele dasselbe Settings-Objekt erhalten, können Werte zwischen den Spielen erhalten bleiben.

#### Gemeinsame Einstellungen und Spielwerte

```
self.screen_width = 800  
self.screen_height = 600  
self.fps = 60  
self.gold_coins = 0  
self.lives = 3  
self.current_level = 0
```

### 3.4 GameOne und GameTwo

GameOne und GameTwo erben von BaseGame. Beide Spiele verwenden dieselbe Struktur, reagieren aber unterschiedlich auf die Leertaste: In GameOne wird die Anzahl der Goldmünzen erhöht, in GameTwo verringert. Dadurch wird sichtbar, dass beide Spiele auf denselben gemeinsamen Wert zugreifen.

#### Unterschiedliche Spiellogik bei gleicher Grundstruktur

```
# GameOne
if event.key == pygame.K_SPACE:
    self.settings.gold_coins += 1

# GameTwo
if event.key == pygame.K_SPACE:
    self.settings.gold_coins -= 1
```

## 4. Ablauf des Spielwechsels

Der Spielwechsel wird im aktuellen Stand zentral über die ENTER-Taste gesteuert. Die Hauptschleife sammelt zunächst alle Pygame-Events. Wenn ein KEYDOWN-Event mit K\_RETURN erkannt wird, wird `handle_enter_key()` aufgerufen. Diese Methode erhöht den aktuellen Spielindex. Ist das Ende der Liste erreicht, springt der Index wieder auf 0 zurück.

### Spielwechsel per Index

```
def handle_enter_key(self):
    self.current_game_index += 1
    if self.current_game_index >= len(self.game_list):
        self.current_game_index = 0
    self.switch_game(self.current_game_index)
```

Danach wird in jedem Schleifendurchlauf das aktuell aktive Spiel aus der Liste geholt. Nur dieses Spiel erhält die Events, wird aktualisiert und gezeichnet.

### Aufruf des aktiven Spiels

```
current_game = self.game_list[self.current_game_index]
current_game.handle_events(events)
current_game.update()
current_game.draw()
```

|                        |                        |                         |                               |                                      |
|------------------------|------------------------|-------------------------|-------------------------------|--------------------------------------|
| <b>1. Events lesen</b> | <b>2. ENTER prüfen</b> | <b>3. Index erhöhen</b> | <b>4. Aktives Spiel holen</b> | <b>5. Zeichnen im selben Fenster</b> |
|------------------------|------------------------|-------------------------|-------------------------------|--------------------------------------|

## 5. Bewertung der Paketstruktur und `__init__.py`

Der Ordner `games/` ist als Package gedacht. Die Datei `__init__.py` ist bei modernen Python-Versionen nicht immer zwingend notwendig, sie ist in Schul- und Projektstrukturen aber weiterhin sinnvoll, weil sie eindeutig zeigt: Dieser Ordner gehört als Python-Paket zur Anwendung.

Wichtig ist jedoch, dass die Datei sauber formuliert wird. Die aktuelle Idee, Klassen in `__init__.py` bereitzustellen, ist grundsätzlich möglich. Korrekt wäre dabei ein relativer Import aus den Modulen im eigenen Package.

### Empfohlene Variante für `__init__.py`

```
# games/__init__.py
from .game_one import GameOne
from .game_two import GameTwo
# from .game_three import GameThree
```

**Hinweis:** Die Zeile `from games import GameOne, GameTwo` innerhalb der `__init__.py` ist problematisch, weil sich das Package damit gewissermaßen selbst importiert. Besser sind relative Importe mit Punkt: `from .game_one import GameOne`.

Mit dieser Variante könnte `main.py` kürzer importieren:

### Kürzerer Import nach sauberem `__init__.py`

```
from games import GameOne, GameTwo
```

Alternativ kann main.py weiterhin direkt aus den einzelnen Modulen importieren. Das ist ebenfalls korrekt und für Anfänger oft transparenter:

#### Direkter Import aus den Modulen

```
from games.game_one import GameOne
from games.game_two import GameTwo
```

## 6. Erweiterbarkeit des Projekts

Das Projekt lässt sich gut erweitern, weil jedes neue Spiel nur die gemeinsame Basisklasse erben und die drei Standardmethoden implementieren muss. Der Manager muss anschließend nur das neue Spiel importieren und in game\_list ergänzen.

#### Beispiel für ein weiteres Spiel

```
# games/game_three.py
from base_game import BaseGame

class GameThree(BaseGame):
    def handle_events(self, events):
        pass

    def update(self):
        pass

    def draw(self):
        self.screen.fill((30, 120, 80))
```

#### Ergänzung in main.py

```
from games.game_three import GameThree

self.game_list = [
    GameOne(self.screen, self.settings),
    GameTwo(self.screen, self.settings),
    GameThree(self.screen, self.settings),
]
```

## 7. Adaptivität

| Verbesserung                    | Nutzen                                                                                                                                                     |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dynamischer Spielwechsel</b> | Die im BaseGame vorhandenen Attribute active und next_game könnten genutzt werden, damit ein Spiel selbst signalisiert, welches Spiel danach starten soll. |
| <b>Startbildschirm / Menü</b>   | Statt ENTER direkt als globalen Wechsel zu verwenden, könnte ein Menü mit Buttons oder Auswahlpunkten eingeführt werden.                                   |
| <b>Reset-Methoden</b>           | Falls ein Spiel beim erneuten Öffnen neu starten soll, könnte jede Spielklasse eine reset()-Methode erhalten.                                              |

## 8. Zusammenfassung

**Thema:** Wechsel zwischen einzelnen Spielen innerhalb eines Pygame-Fensters und sinnvolle Paketstruktur

### Grundidee

- Mehrere Mini-Spiele laufen in einem gemeinsamen Pygame-Fenster.
- Ein zentraler Manager entscheidet, welches Spiel gerade aktiv ist.
- Alle Spiele erben von einer gemeinsamen Basisklasse BaseGame.
- Gemeinsame Werte wie Gold, Leben und FPS liegen in SuperSettings.
- Der Wechsel erfolgt aktuell mit der ENTER-Taste.

### Aufbau des Projekts

| Bestandteil               | Bedeutung                                                                          |
|---------------------------|------------------------------------------------------------------------------------|
| <code>main.py</code>      | Startet Pygame, erstellt den Manager und enthält die Hauptschleife.                |
| <code>settings.py</code>  | Speichert globale Einstellungen und gemeinsame Spielwerte.                         |
| <code>base_game.py</code> | Definiert die gemeinsame Schnittstelle für alle Spiele.                            |
| <code>games/</code>       | Enthält die einzelnen Spielmodule.                                                 |
| <code>__init__.py</code>  | Kennzeichnet <code>games/</code> als Package und kann optionale Exporte enthalten. |

### Wichtiger Ablauf in der Hauptschleife

1. Events werden einmal zentral mit `pygame.event.get()` gesammelt.
2. Bei ENTER wird `current_game_index` erhöht.
3. Der Index springt nach dem letzten Spiel wieder auf 0.
4. Nur das aktive Spiel bekommt `handle_events()`, `update()` und `draw()`.
5. `pygame.display.flip()` aktualisiert das gemeinsame Fenster.

### Merksatz

**Merksatz:** Der Manager verwaltet das Fenster und entscheidet, welches Spiel aktiv ist. Die Spiele selbst kennen nur ihre eigene Logik und Darstellung.

### Saubere `__init__.py`

Empfohlen: entweder leer lassen oder relative Exporte nutzen, z. B. `from .game_one import GameOne`.

### Fazit

Die Struktur ist übersichtlich und leicht erweiterbar. Entscheidend ist die klare Trennung zwischen Manager, Einstellungen, Basisklasse und einzelnen Spielmodulen.